# Payroll AI: technical documentation

A payroll system for a small team. It keeps the employee directory, calculates monthly pay with Philippine deductions, and uses AI to flag unusual payroll runs. It is built as three services and a web app.

Version 1.0 Date 29 October 2026 Status Capstone project, runs locally

Audience Reviewers and developers

## Summary

Payroll AI has four parts: a React web app and three backend services written in Java, Node.js and Python. Each service has one job. The web app calls them directly over HTTP and sends JSON. A single signed token, issued at sign-in, proves who the user is to every service.

This document explains what problem the system solves, what is in and out of scope, how the parts fit together, what each part is responsible for, how a request flows through the system, which technologies are used, and what should be improved next. Everything described here was checked against the project’s source code.

## 1. The problem it solves

Small teams often run payroll from spreadsheets. Salaries are copied by hand, Philippine contributions and withholding tax are recalculated every month, the same person can be paid twice for one month, anyone with the file can change anything, and odd figures are noticed only after payment. Payroll AI replaces that routine with one system.

| Problem with manual payroll | How Payroll AI addresses it |
|---|---|
| Salaries live in several places and drift apart | The employee directory is the single source. Payroll reads the salary from it at run time and never accepts a salary from the browser. |
| Contributions and tax are recalculated by hand | The payroll engine adds overtime and calculates SSS, PhilHealth, Pag-IBIG and withholding tax automatically. |
| Duplicate payments | Only one payroll run can exist for an employee and month. A second attempt is rejected. |
| No control over who can change data | Sign-in with two roles. Admins change data. Members can view, export and ask questions. |
| Mistakes are found too late | An AI review applies fixed rules and an OpenAI check, and marks runs that need a human to look. |

| Problem with manual payroll | How Payroll AI addresses it |
|---|---|
| Questions need someone to dig through records | A chat assistant answers questions about employees and payroll, using only the live data. |
| Hard to identify people in a list | Admins can attach a photo to each employee, stored privately in AWS S3. |

> The tax and contribution rates in the code are demo values. Confirm them with SSS, PhilHealth, Pag-IBIG and the BIR before using the system for real payroll.

# 2. Scope

## In scope

- Sign-up, sign-in, forgot and reset password, with admin and member roles
- Employee directory: list, view, add, delete, with validation
- Monthly payroll run for one employee at a time, with a payslip and CSV export
- Overtime pay and four Philippine deductions
- Employee photos in AWS S3
- AI review of saved payroll runs
- Chat assistant limited to employee and payroll data
- Local deployment on `localhost` with demo data

## Out of scope

- Permanent storage of employees and accounts (they are kept in memory)
- Editing an employee record (only add and delete exist)
- Pay periods other than one calendar month
- 13th-month pay, leave, loans, holiday and night-shift rules
- Tax filing, bank transfers and e-mailing payslips
- E-mail delivery of password reset codes
- HTTPS, multiple companies, and production hosting
- Legal certification of the tax and contribution rates

The intended users are a payroll officer (admin) and colleagues who only need to see the data (members). The system starts with 10 sample employees and one administrator account created at startup. Change that account’s default password before any real use.

# 3. System architecture

Payroll AI follows a microservice style. The web app talks to each service directly, with no gateway between them. The services talk to each other with HTTP requests and JSON. They do not share a database or any code.

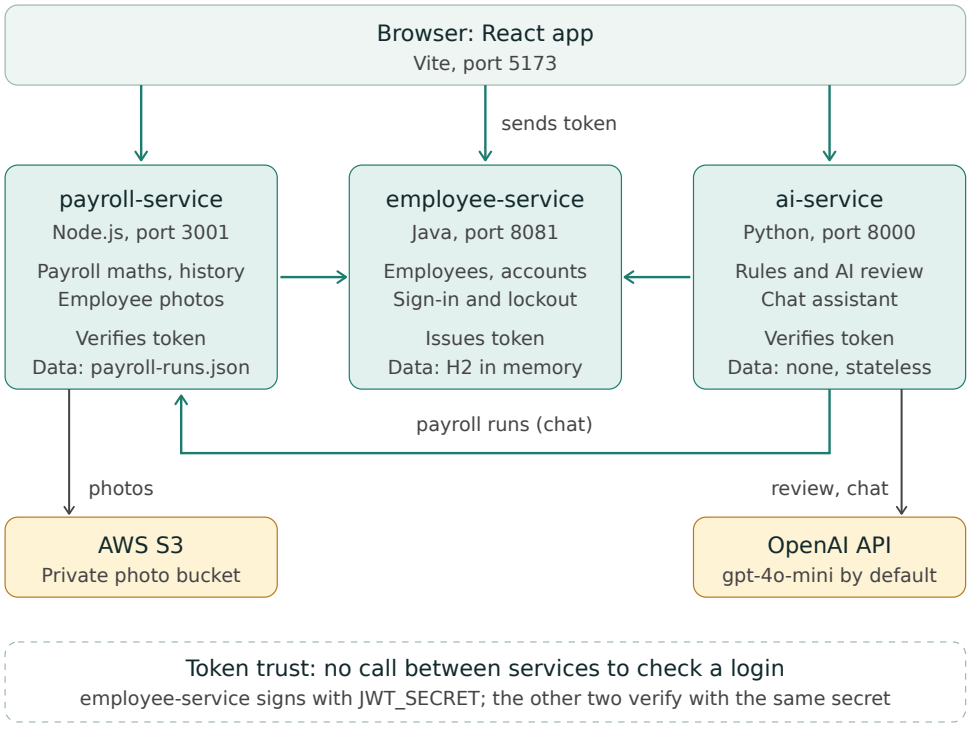



Figure 1. Components, their connections and where each one keeps data.
Solid teal arrows are service calls; grey arrows go to outside systems.

## Design principles

- One job per service, so each can change without touching the others.
- Services agree on messages (JSON field names) and on the token format, not on technology. That is why three languages can work together.
- Tokens are verified locally in every service with a shared secret, so no service has to call another to check a login.
- The server is the source of truth. The salary used for payroll always comes from employee-service, never from the browser.
- Features degrade gracefully. If OpenAI is down, the rule checks still work. If S3 is off, the app works without photos.

## Layers inside each service

The three services use the same four layers, in different languages. A request flows down through the layers and the answer flows back up.

| Layer | Purpose | employee-service | payroll-service | ai-service |
|---|---|---|---|---|
| Gate | Checks the token and role first | `AuthInterceptor` | auth middleware | `current_token` dependency |
| Entry | Receives the request, returns the response | Controllers | Routes and controllers | API routers |
| Rules | The real work | Services | Services | Services |
| Data | Stores or fetches data | Repositories (H2) | Repositories (JSON file, S3) | Data source (HTTP) and OpenAI client |

## 4. Components and responsibilities

Access levels used below: **Public** means no token is needed, **Signed in** means any valid token, and **Admin** means a valid token with the admin role.

### Frontend (React) Port 5173

**Role:** the only part the user sees. It collects input, calls the services and shows the results.

#### Responsibilities

- Sign-in, sign-up and password reset screens. The token is kept in `sessionStorage` and the user is signed out when it expires.
- Employee table, add-employee form, photo upload, run-payroll form, payroll history, payslip view and CSV download.
- Instant form checks (name, `@pwc.com` email, `+63` phone, salary range per position). These help the user but are not the real protection, because the services check again.
- The AI review button and the floating chat assistant.

#### How it is built

The app is one file, `src/App.jsx`, with function components: `Root` (chooses Login or the app), `Login`, `App` (main page), `ChatWidget`, `Payslip` and `Icon`. A single `request()` helper adds the `Authorization` header to every call and turns error replies into messages. There is no router library. The sidebar links scroll to sections of the same page.

### employee-service (Spring Boot) Java 21, port 8081

**Role:** the employee directory and the identity provider. It is the only service that issues tokens.

#### Responsibilities

- Create and read employee records, with validation in Java and as database constraints.

- Create accounts, store only salted password hashes (PBKDF2-SHA512, 210,000 iterations), and sign users in.
- Issue a signed token (HS256, valid 60 minutes) carrying the user id, email and role.
- Lock an account for 15 minutes after 5 wrong passwords.
- Create a 6-digit reset code valid for 15 minutes. No e-mail service exists, so the code is written to the service log.
- Create the first administrator at startup, and load 10 sample employees.

### Interfaces

| Method and path | Access | Purpose |
|---|---|---|
| `POST /api/auth/signup` | Public | Create a member account. Only `@pwc.com` addresses are accepted. |
| `POST /api/auth/login` | Public | Check the password and return the token and its expiry time. |
| `POST /api/auth/forgot-password` | Public | Create a one-time reset code. |
| `POST /api/auth/reset-password` | Public | Set a new password using the code. |
| `GET /api/employees` | Signed in | List all employees. |
| `GET /api/employees/{id}` | Signed in | Get one employee. Used by payroll-service to read the salary. |
| `POST /api/employees` | Admin | Add an employee. Returns 201, or 400 with field errors, or 409 for a duplicate email. |
| `DELETE /api/employees/{id}` | Admin | Delete an employee. Returns 204. |

### Data and structure

An in-memory H2 database with three tables: `employees`, `users` and `password_resets`. Employee rules enforced by the database include a unique lowercase `@pwc.com` email, a phone number starting with `+63`, and a salary that fits the position (Associate 20,000 to 60,000; Senior Associate above 60,000 up to 100,000; Manager above 100,000). Code packages: `controller`, `service`, `repository`, `security` (`JwtService`, `PasswordHasher`, `AttemptTracker`, `AuthInterceptor`), `dto`, `model` and `config`. This is the only service with automated tests in the repository.

**Depends on:** nothing. Other services call it.

## payroll-service (Express) Node.js, port 3001

**Role:** payroll calculation, payroll history and employee photos.

## Responsibilities

- Validate a payroll request, fetch the real salary from employee-service, calculate pay and save the run.
- Allow only one run per employee and month (409 Conflict on a repeat).
- Return the history and export one run as a CSV file.
- Store, list and delete employee photos in a private S3 bucket and return links that expire after 5 minutes.
- Verify the token on every request under `/api` and enforce admin-only actions.

## Interfaces

| Method and path | Access | Purpose |
|---|---|---|
| `GET /health` | Public | Returns `{"status":"ok"}`. |
| `GET /api/payroll` | Signed in | List all saved payroll runs. |
| `POST /api/payroll/run` | Admin | Run payroll for an employee and month. Returns 201. |
| `GET /api/payroll/{id}/export` | Signed in | Download one run as CSV. |
| `GET /api/employee-images` | Signed in | Map of employee id to a temporary photo link. |
| `PUT /api/employee-images/{employeeId}` | Admin | Upload a photo (raw JPEG, PNG or WebP, up to 2 MB). |
| `DELETE /api/employee-images/{employeeId}` | Admin | Remove a photo. |
| `GET /api/aws/identity` | Admin | Show which AWS account is in use, with the account number partly hidden. |

## Payroll calculation

Overtime pay is `base salary / 160 × 1.25 × overtime hours`, and gross pay is base salary plus overtime pay. Contributions are based on the basic salary. Withholding tax is based on gross pay minus those contributions. The request must have a positive employee id, a period such as `2026-10`, overtime between 0 and 100 hours, and non-negative extra deductions. Total deductions cannot exceed gross pay.

| Item | Rule as coded |
|---|---|
| SSS | 5% of the salary credit: salary rounded to the nearest 500, with a minimum of 5,000 and a maximum of 35,000. |
| PhilHealth | 2.5% of salary, with the salary limited to between 10,000 and 100,000. |
| Pag-IBIG | 1% when salary is 1,500 or less, otherwise 2% of salary capped at 10,000. |

| Item | Rule as coded |
|---|---|
| Withholding tax | Monthly brackets on taxable income. Up to 20,833: 0. Above 20,833: 15% of the excess. Above 33,333: 1,875 plus 20%. Above 66,667: 8,541.80 plus 25%. Above 166,667: 33,541.80 plus 30%. Above 666,667: 183,541.80 plus 35%. |

Worked example: a salary of 30,000 with 10 overtime hours gives overtime pay of 2,343.75 and gross pay of 32,343.75. Contributions are 1,500 + 750 + 200 = 2,450. Taxable income is 29,893.75 and tax is 1,359.11. Total deductions are 3,809.11, so net pay is 28,534.64.

### Data and structure

Payroll runs are stored in `data/payroll-runs.json`. Writes go through a queue and are saved to a temporary file that is then renamed, so a crash cannot leave a half-written file. Photos are stored in S3 under the key `employees/{id}/photo`. Folders: `routes`, `controllers`, `middleware`, `services` (`payroll`, `ph-deductions`, `image`, `aws`, `employee.client`), `repositories`, `config`.

**Depends on:** employee-service (salary lookup and employee check) and AWS S3.

## ai-service (FastAPI) Python, port 8000

**Role:** a second opinion on payroll runs, and a chat assistant that answers from live data. It stores nothing.

### Responsibilities

- Apply three fixed review rules to each payroll run: overtime above 24 hours, deductions above 20% of gross pay, and gross pay minus deductions not matching net pay.
- Send six numeric fields per run (no names) to OpenAI in batches of 50 and collect a flag and a short reason for each run.
- Mark a run “review” if a rule or the AI flags it. If OpenAI fails, return the rule results with `ai_available` set to false.
- For chat, load employees and payroll runs from the other two services using the user’s own token, and ask OpenAI to answer only from that data.
- Verify the token on every `/api/ai` request.

### Interfaces

| Method and path | Access | Purpose |
|---|---|---|
| `GET /health` | Public | Returns the status and whether the mode is `openai` or `rules-only`. |
| `POST /api/ai/analyze` | Signed in | Body `{"records":[…]}`. Returns a status, flag, reason and warnings for each run. |
| `POST /api/ai/chat` | Signed in | Body `{"messages":[…]}`. Returns `{"answer":"…"}`. |

### Limits and structure

The chat accepts up to 12 messages of 1,000 characters each, sends at most the latest 200 payroll runs, uses temperature 0 and waits at most 5 seconds for each data call. OpenAI calls time out after 30 seconds. The model defaults to `gpt-4o-mini` and can be changed in `.env`. Folders: `api`, `core` (`config`, `auth`), `schemas` and `services` (`analysis`, `chat`, `data_source`, `openai_review`, `openai_client`).

**Depends on:** employee-service and payroll-service (chat only) and the OpenAI API.

## 5. System flow

Every message between parts is an HTTP request with a JSON body, and the token travels in the `Authorization: Bearer` header.

### 5.1 Sign in

1. The browser sends `POST /api/auth/login` with the email and password to employee-service.
2. employee-service checks the lockout counter, compares the password with the stored hash and, if correct, returns a signed token with its expiry time.
3. The app saves the token in `sessionStorage` and shows the main page. A timer signs the user out when the token expires.
4. Every later call carries the token. Each service checks the signature, expiry, issuer and audience itself. A bad token gets 401 and a non-admin using an admin action gets 403.

### 5.2 Run payroll

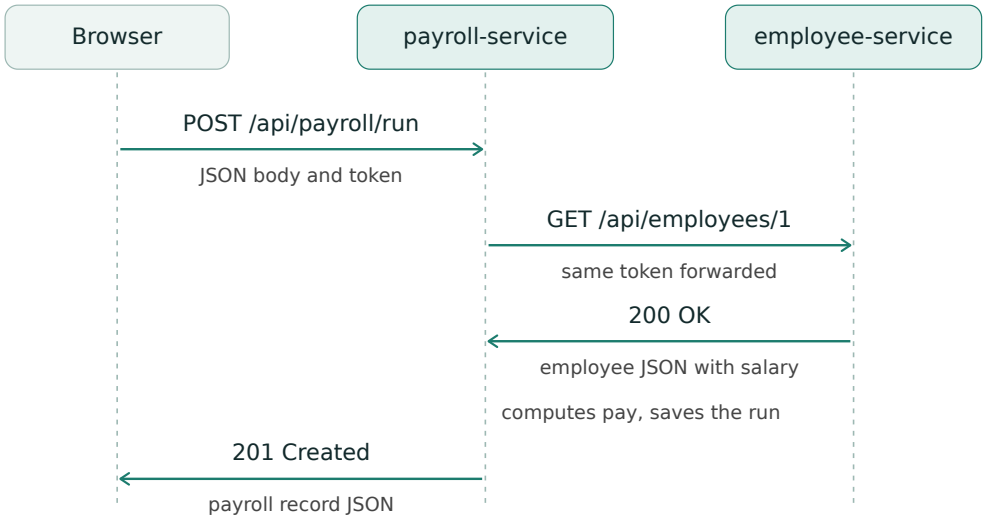



Figure 2. The JSON messages exchanged when an admin runs payroll.

Example request and the start of the reply:

```
POST /api/payroll/run
{ "employeeId": 1, "period": "2026-10", "overtimeHours": 10, "deductions": 0 }

201 Created
{ "employeeId": 1, "fullName": "Ana Cruz", "period": "2026-10",
  "grossPay": 32343.75, "deductions": 3809.11, "netPay": 28534.64, … }
```

1. The browser sends the employee, month, overtime hours and extra deductions.
2. payroll-service checks the input, then asks employee-service for the employee. If employee-service is down it returns 502.
3. It calculates overtime, contributions, tax, total deductions and net pay.
4. It saves the run only if none exists for that employee and month. Otherwise it returns 409.
5. The browser shows the payslip. The history list reloads.

### 5.3 Upload an employee photo

1. The browser checks the file type and size, then sends `PUT /api/employee-images/{id}` with the raw image.
2. payroll-service accepts only admins, limits the body to 2 MB, and reads the file’s first bytes to confirm it is a JPEG, PNG or WebP image.
3. It asks employee-service whether the employee exists, then stores the file in S3.
4. It returns a link that works for 5 minutes. The app reloads the photo list and shows the picture straight from S3.

### 5.4 AI review

1. The browser sends the saved payroll runs to `POST /api/ai/analyze`. ai-service does not fetch them itself.
2. ai-service checks the shape of every record and applies the three fixed rules.
3. It sends six numeric fields per run to OpenAI and receives a flag and reason for each.
4. It returns the combined result. A run needs a human look if a rule or the AI flagged it. The reply always includes a notice that the review is not a fraud or statutory finding.

### 5.5 Chat assistant

1. The browser sends the conversation to `POST /api/ai/chat`.
2. ai-service calls `GET /api/employees` on employee-service and `GET /api/payroll` on payroll-service, with the user’s token. If one fails, it answers from the other and says the missing section could not be loaded.
3. It sends the data and the conversation to OpenAI with instructions to answer only from the data, and to decline other topics.
4. It returns the answer text.

### 5.6 Error handling

| Situation | Response |
|---|---|
| Missing, forged or expired token | 401, and the app signs the user out |
| Valid token but not an admin | 403 |
| Invalid input | 400, with `fieldErrors` for forms |
| Employee does not exist | 404 |
| Payroll already exists for that employee and month, or duplicate email | 409 |
| Photo larger than 2 MB | 413 |
| Too many wrong passwords | 429, locked for 15 minutes |
| employee-service unavailable during payroll or photo upload | 502 |
| S3 off or unreachable | 503 for photo features; the rest works |
| OpenAI missing or failing | Review: rule results with `ai_available: false`. Chat: 503 or 502. |

Errors are returned as JSON, for example `{"error":"Payroll already exists for this employee and period"}`.

## 6. Technical stack

| Part | Technology | Used for |
|---|---|---|
| Frontend | React 19, Vite, plain `fetch`, CSS, ESLint | User interface and calls to the services |
| employee-service | Java 21, Spring Boot 4.1 (web, JDBC, validation), H2 in-memory database, Nimbus JOSE JWT, Maven wrapper | Employees, accounts, token issuing |
| payroll-service | Node.js, Express 5, `jsonwebtoken`, `cors`, `dotenv`, AWS SDK v3 (S3, presigner, STS) | Payroll maths, history, photos |
| ai-service | Python 3.11 or 3.12, FastAPI, Uvicorn, Pydantic, `httpx`, OpenAI SDK, PyJWT, `python-dotenv` | Rule checks, AI review, chat |
| Outside services | AWS S3 and STS, OpenAI API (default model `gpt-4o-mini`) | Photo storage, AI features |
| Communication | HTTP and REST, JSON, JWT signed with HS256 | All messages between parts |
| Storage | H2 (memory), JSON file, S3 bucket | Employees and accounts, payroll runs, photos |

### Configuration

Each service reads a local `.env` file that is ignored by git. A template named `.env.example` shows the names.

| Setting | Service | Meaning |
|---|---|---|
| `JWT_SECRET` | All three | At least 32 characters and identical in all three services |
| `EMPLOYEE_SERVICE_URL` | payroll, ai | Where employee-service runs (default `http://localhost:8081`) |
| `PAYROLL_SERVICE_URL` | ai | Where payroll-service runs (default `http://localhost:3001`) |
| `OPENAI_API_KEY`, `OPENAI_MODEL` | ai | OpenAI access and model name |
| `AWS_ENABLED`, `AWS_REGION`, `AWS_S3_BUCKET`, `AWS_PROFILE` | payroll | Turn photo storage on, and choose the region, bucket and AWS login |
| `PORT` | payroll | Port for payroll-service (default 3001) |

### Running locally

Copy each `.env.example` to `.env` and fill in the values, using the same `JWT_SECRET` everywhere. Start employee-service first, because the others call it.

```
# employee-service (port 8081)
./mvnw spring-boot:run

# payroll-service (port 3001)
npm install
npm start

# ai-service (port 8000)
python -m venv .venv
python -m pip install -r requirements.txt
python -m uvicorn app.main:app --port 8000

# frontend (port 5173)
npm install
npm run dev
```

## 7. Security and operations

| Area | How it is handled |
|---|---|
| Passwords | Never stored. Only salted PBKDF2-SHA512 hashes are kept, compared in constant time. New passwords need at least 8 characters. |
| Tokens | HS256, 60 minutes. Each service accepts only HS256 and checks signature, expiry, issuer and audience. |
| Authorization | Role is read from the token. Admin-only actions return 403 for members. |
| Brute force | Five wrong passwords lock the account for 15 minutes. |
| Input validation | Checked in the browser, in each service, and for employees also by database constraints. |

| Area | How it is handled |
|---|---|
| Cross-origin access | Every service allows browser calls only from `http://localhost:5173`. |
| Secrets | Kept in `.env` files, not in code. The OpenAI key is used only inside ai-service. |
| Data sent to OpenAI | Review sends six numbers per run. Chat also sends employee names, emails, phone numbers and salaries. |
| Photos | The type is checked from the file's bytes, the size is limited to 2 MB, the bucket is private and links expire after 5 minutes. |

### Known limitations

- Employees and accounts are kept in memory. A restart removes every account created by sign-up and resets employee ids, so saved payroll runs and photos could point to a different person.
- A token cannot be cancelled before it expires. Signing out only makes the browser forget it.
- One shared secret signs every token. Anyone who learns it can create an admin token.
- Traffic is plain HTTP on localhost.
- The "deductions above 20% of gross" review rule can flag correctly calculated pay for higher-paid managers, because the tax brackets put their deductions near 20% to 22% of gross.
- The duplicate-payroll check protects a single running payroll-service. Two instances could both pass it.
- Some project documents are out of date. For example, the `ai-service` README describes an offline scikit-learn model that the code does not use. The service now uses fixed rules and OpenAI.

## 8. Future enhancements

Ordered by priority. Each item answers a limitation or gap described above.

| Priority | Enhancement | Why |
|---|---|---|
| High | Move employees and accounts to PostgreSQL | Stops data loss on restart and the reuse of employee ids. |
| High | Store payroll history in a database with a unique constraint on employee and month | Makes the one-run-per-month rule safe with several instances. |
| High | HTTPS between all parts, and a secrets manager with a rotated `JWT_SECRET` | Protects tokens and keys on the network and in files. |
| High | Automated tests for payroll maths, validation, the AI rules and sign-in | Only employee-service has tests today. |
| Medium | Refresh tokens with revocation, and asymmetric signing (RS256) | Allows cancelling a token, and removes the need to share a signing secret. |
| Medium | E-mail delivery for password reset codes | Codes are written to a log today. |

| Priority | Enhancement | Why |
|---|---|---|
| Medium | Edit employee records | Only add and delete exist. |
| Medium | Review rules tuned by salary band | Removes false alarms for managers. |
| Medium | Mask personal data before it goes to OpenAI, and review privacy requirements | Chat sends names, emails, phone numbers and salaries. |
| Medium | Delete photos on the server when an employee is deleted | The browser does it today and ignores failures. |
| Later | Bulk payroll runs, PDF payslips, and more pay components (13th month, holiday pay) | Closer to real payroll practice. |
| Later | A shared API description (OpenAPI) and an API gateway | Makes the JSON contract between services explicit and gives one entry point. |
| Later | Split `App.jsx` into component files and add a router | Easier to maintain. |
| Later | Containers and Docker Compose, a build pipeline, structured logs and an audit trail | Easier deployment, monitoring and accountability. |

# Glossary

| Term | Meaning |
|---|---|
| Microservice | A small program that does one job and talks to others over a network. |
| REST and JSON | A style of web interface where URLs name things and the method says the action, with data sent as JSON text. |
| JWT | JSON Web Token: a signed pass that carries the user’s id, email, role and expiry. It can be read by anyone but not changed without breaking the signature. |
| CORS | A browser rule that decides which web pages may call a given server. |
| H2 | A small database that here lives in memory and is wiped on restart. |
| Signed URL | A temporary link to a private file that stops working after a few minutes. |
| SSS, PhilHealth, Pag-IBIG, BIR | The Philippine agencies for social security, health insurance, housing fund and tax. |